1) 핵심 분류(대표 CWE/OWASP 항목)

- **인젝션(Injection)**: SQL/OS/LDAP/NoSQL 주입, 템플릿/명령어 주입. 사용자 입력이 쿼리/명령에 그대로 섞일 때. (CWE-89, CWE-77)
- **인증/인가 결함**: 취약한 로그인, 토큰 검증 누락, 권한 체크 누락(=IDOR). (CWE-287, CWE-639)
- **XSS/CSRF**: 스크립트 주입, 요청 위조로 사용자의 권한으로 동작. (CWE-79, CWE-352)
- **SSRF**: 서버가 내부망/메타데이터 엔드포인트에 요청을 보내도록 유도. (CWE-918)
- **경로/파일 조작**: ../ 등으로 임의 파일 접근/업로드. (CWE-22)
- **원격 코드 실행/역직렬화**: 위험 API(eval, pickle, 취약한 Java 역직렬화 등). (CWE-94, CWE-502)
- **암호화/랜덤 취약**: MD5/SHA-1, ECB, 고정 시드/예측 가능한 난수. (CWE-327, CWE-330)
- **민감정보 노출**: 로그/응답/리포지토리에 키·토큰·개인정보 노출. (CWE-200, CWE-522)
- **구성 오류(Misconfiguration)**: 디버그 모드, 과도한 CORS, 기본 비밀번호, S3 공개. (CWE-16)
- **의존성 취약**: 취약 라이브러리/프레임워크 버전 사용(공개 CVE). (CWE-1104)

2) 언어/스택별 자주 나오는 패턴

- **Java/Spring**: 위험한 역직렬화(ObjectInputStream), 개방형 CORS, SpEL/템플릿 주입, 파일 업로드 검증 부재.
- **Python**: eval/exec, pickle.loads, 서드파티 명령 실행(subprocess), 디버그 서버 공개, SQL 구성 문자열 포매팅.
- **Node.js**: child_process, 템플릿 주입(ejs/handlebars), NoSQL 인젝션, 위험한 serialize-javascript, 정적 파일 공개 범위 과다.

3) "취약점의 징후" 체크리스트(코드 리뷰/정적분석용)

- 입력값이 **검증·정규화 없이** 쿼리/명령/파일경로에 결합된다.
- 권한 확인이 **엔드포인트/서비스 계층**에서 일관되지 않다.
- 인증 토큰/비밀키가 **하드코딩**되어 있거나 로그로 남는다.
- 암호화 알고리즘/모드/키관리 방식이 **구식**이거나 **직접 구현**했다.
- **에러 메시지/스택 트레이스**가 외부로 누출된다.
- 기본 설정이 "허용(allow)" 위주(CORS *, S3 public, 디버그 True 등).
- 종속성 버전이 **오래되었거나** 보안 공지로 막힌 버전.

4) 빠른 우선순위 매기기(Severity = 영향 × 가능성)

- CRITICAL: RCE, 인증우회, 대규모 데이터 유출 가능 (즉시 조치).
- HIGH: SQLi/XSS/SSRF/권한상승 등 실질적 공격 경로.
- MEDIUM: 정보 노출, 취약 구성, 취약 라이브러리(악용 조건 제한).
- LOW: 모범사례 미준수, 보수적 개선 권장 영역.

5) 안전한 대처 패턴(핵심만)

- **입력 검증 & 인코딩**: SQL 은 **파라미터 바인딩**, HTML 은 **출력 인코딩**.
- **권한/소유권 검사 표준화**: 미들웨어/어노테이션으로 **중앙집중 권한 체크**.
- **비밀 관리**: 코드/깃이 아니라 **시크릿 매니저/환경변수**.
- **안전한 크립토**: 최신 알고리즘(예: AES-GCM, Argon2), 키는 KMS/HSM.
- **의존성 관리**: 버전 고정/주기적 업데이트, SCA 도구 병행.
- **안전한 기본값**: 최소권한, 폐쇄형 CORS, 디버그 OFF, 디렉토리 인덱싱 차단.
- **로깅/모니터링**: 민감정보 마스킹, 이상행위 탐지.

6) 단초 코드 예시(취약 → 안전)

SQL 인젝션 (Python)

```
# ❌ 취약
cur.execute(f"SELECT * FROM users WHERE name = '{user_input}'")
# ☑ 안전
cur.execute("SELECT * FROM users WHERE name = %s", (user_input,))
```

경로 조작 (Java)

```
// ❌ 취약
Path p = Paths.get(uploadDir, filename); Files.copy(in, p);
// ☑ 안전(화이트리스트/정규화)
Path p = Paths.get(uploadDir).resolve(filename).normalize();
if (!p.startsWith(uploadDir)) throw new SecurityException();
```

민감정보 노출 (Node)

```
// ❌ 취약
console.log("TOKEN:", process.env.API_TOKEN)
// ☑ 개선
console.log("Auth configured"); // 토큰은 로그에 남기지 않기
```